

Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Parts:

- Part 1: Complete 1 of 3 questions
- Part 2: Review your partner's Assignment 1 submission
- Part 3: Perform code review of your partner's assignment 1 by answering the questions below
- Part 3: Reflect on Assignment 1 and Assignment 2

Part 1:

You will be assigned one of three problems based of your first name. Enter your first name, in all lower case, execute the code below, and that will tell you your assigned problem. Include the output as part of your submission (do not clear the output). The problems are based-off problems from Leetcode.

```
In [1]: import hashlib

def hash_to_range(input_string: str) -> int:
    hash_object = hashlib.sha256(input_string.encode())
    hash_int = int(hash_object.hexdigest(), 16)
    return (hash_int % 3) + 1
input_string = "Rajthilak"
result = hash_to_range(input_string)
print(result)

2

• Question 1
```

Starter Code for Question 1

```
In [ ]: # Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, val = 0, left = None, right = None):
#         self.val = val
#         self.left = left
#         self.right = right
def is_duplicate(root: TreeNode) -> int:
    # TODO

• Question 2
```

Starter Code for Question 2

```
In [3]: from typing import List, Optional

# Definition for a binary tree node.
class TreeNode(object):
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
        self.right = right
def bt_path(root: TreeNode) -> List[List[int]]:
    def dfs(node, path):
        if not node:
            return
        path.append(node.val)
        if not node.left and not node.right: # If it's a leaf node
            paths.append(path[:]) # Add a copy of the path
        else:
            dfs(node.left, path)
            dfs(node.right, path)
        path.pop() # Backtrack

    paths = []
    dfs(root, [])
    return paths

# Helper function to create a binary tree from a list (level-order input)
def build_tree(values: List[Optional[int]]) -> Optional[TreeNode]:
    if not values:
        return None
    nodes = [TreeNode(val) if val is not None else None for val in values]
    kids = nodes[:-1]
    root = kids.pop()
    for node in nodes:
        if node:
            if kids:
                node.left = kids.pop()
            if kids:
                node.right = kids.pop()
    return root

# Example usage
example1 = build_tree([1, 2, 2, 3, 5, 6, 7])
example2 = build_tree([10, 9, 7, 8])

print("Example 1 Output:", bt_path(example1))
print("Example 2 Output:", bt_path(example2))

Example 1 Output: [[1, 2, 3], [1, 2, 5], [1, 2, 6], [1, 2, 7]]
Example 2 Output: [[10, 9, 8], [10, 7]]

• Question 3
```

Starter Code for Question 3

```
In [ ]: def missing_num(nums: List) -> int:
# TODO
```

Part 2:

You and your partner must share each other's Assignment 1 submission.

Part 3:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

The give input should be kept in original order but all the zeroes should be moved to the end. But the non-zero should be kept in the same order as it is in the input.

- Create 1 new example that demonstrates you understand the problem. Trace/walkthrough 1 example that your partner made and explain it.

Input: nums = [0, 8, 0, 3, 9]

Output: [8, 3, 9, 0, 0]

The give input non-numeric numbers 8,3,9 are put in the same order as it is in the input, and whatever number of zeroes left are appended to the end.

- Copy the solution your partner wrote.

```
In [5]: from typing import List

def move_zeros_to_end(nums: List[int]) -> List[int]:
    for i,num1 in enumerate(nums):
        if num1==0:
            for j,num2 in enumerate(nums[i+1:]):
                if num2!=0:
                    nums[i],nums[j+i+1]=nums[j+i+1],nums[i]
                    break
    return nums

move_zeros_to_end([0, 1, 0, 3, 12])

Out[5]: [1, 3, 12, 0, 0]
```

- Explain why their solution works in your own words.

The function goes through the list one by one. If the number is 0, it finds the next number that is not 0, after the current position, and swaps the non-zero number to zero number position. By doing this, non-zero numbers are moved forward, and zeros move toward the end.

- Explain the problem's time and space complexity in your own words.

The outer loop goes through each element of the list once so the time complexity O(n). However there is an inner loop, and, in the worst case, for each zero, this can scan the whole remaining list, so another O(n). The resulting time complexity is almost O(n²) (n square). The space complexity is O(1), as it works on the same list without any temps or duplicate lists.

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

I think the only one disadvantage in the given solution could be the time complexity n-square. Having two loops, always increase the time complexity. This should be corrected, the solution should be given in one loop instead of loop within a loop.

Part 4:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

Reflection

Assignment One actually taught me to understand the given problem deeply before jumping into coding. I realized that if we truly grasp the problem, there are multiple ways to solve it — and from there, we can choose the best solution based on time and space complexity. This understanding shifted my focus from just writing working code to writing efficient and maintainable code.

While reviewing my partner's code, I noticed that although it produced the correct output, there were alternative approaches that could improve performance. This review experience helped reinforce the idea that even working code can be optimized, and that there's always room for improvement by adopting better algorithms or patterns.

Sharing our solutions and reviewing each other's work helped me see how different perspectives lead to different implementations. It reminded me that peer feedback is not about right or wrong, but about learning how to think critically and write smarter code. Overall, this assignment strengthened my ability to evaluate problem-solving strategies, and made me more confident in analyzing the trade-offs between correctness, readability, and efficiency.

I look forward to applying this mindset in future assignments and real-world coding challenges.

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

📖 Please review our [Assignment Submission Guide](#) 📖 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: HH:MM AM/PM – DD/MM/YYYY
- The branch name for your repo should be: assignment-2
- What to submit for this assignment:
 - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment: https://github.com/<your_github_username>/algorithms_and_data_structures/pull/<pr_id>
 - Open a private link in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verified that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at [#cohort-6-help](#). Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.

```
In [ ]: 
```